

Chapter 18 - Prepare App to Publish to Render Cloud Service

Chapter 18: Prepare App to Publish to Render Cloud Service

Throughout this course, we have been running FastAPI and Gradio locally.

Development architecture:

```
1 Browser
2   ↓
3 Gradio
4   ↓
5 FastAPI
6   ↓
7 SQLite
8   ↓
9 OpenAI / Claude
10
```

Plain Text ▾

Everything runs on your computer. This is perfect for development, but eventually real users need access to your application.

Instead of:

```
1 http://127.0.0.1:8000
2
```

Plain Text ▾

we want:

```
1 https://your-app.onrender.com
2
```

Plain Text ▾

In this chapter, we will prepare the application for deployment to **Render Cloud Service** and publish our completed AI Business Assistant to the internet.

Learning Objectives

By the end of this chapter, you will be able to:

- Move configuration into a `.env` file
 - Prepare FastAPI for cloud deployment
 - Mount Gradio into FastAPI
 - Run FastAPI and Gradio as a single service
 - Generate a requirements file
 - Push a project to GitHub
 - Deploy a FastAPI application to Render
 - Access your deployed application
 - Update Gradio to use the cloud API
-

1. Why Move Configuration to a .env File?

Currently, values such as:

```
1 OPENAI_API_KEY
2
```

Python ▾

and:

```
1 SECRET_KEY
2
```

Python ▾

may be stored directly inside Python files. For local testing this works.

For cloud deployment, Configuration should be separated from Application Code. This makes configuration easier to manage and more secure.

2. Create a .env File

Create:

```
1 .env
```

Plain Text ▾

Add:

```
1 OPENAI_API_KEY=YOUR-API-KEY
2 SECRET_KEY=my-secret-key
3 ALGORITHM=HS256
4
```

Plain Text ▾

3. Load Environment Variables

Open:

```
1 config/settings.py
```

Plain Text ▾

Install:

```
1 pip install python-dotenv
```

Shell ▾

Update:

Python ▾

```
1 import os
2 from dotenv import load_dotenv
3 load_dotenv()
```

Read values:

Python ▾

```
1 OPENAI_API_KEY = os.getenv(
2     "OPENAI_API_KEY"
3 )
4
5 SECRET_KEY = os.getenv(
6     "SECRET_KEY"
7 )
8
9 ALGORITHM = os.getenv(
10     "ALGORITHM"
11 )
12
```

Now the application reads configuration from:

Plain Text ▾

```
1 .env
```

instead of hardcoded values.

4. Why Gradio and FastAPI Should Run Together

Throughout the course we have been running:

Terminal 1:

Shell ▾

```
1 python -m uvicorn main:app --reload
2
```

Terminal 2:

</> Shell

```
1 python gradio_app.py
2
3
```

Architecture:

Plain Text ▾

```
1 FastAPI
2   ↓
3 Port 8000
```

```
4
5 Gradio
6   ↓
7 Port 7860
```

This works locally.

However, on cloud platforms we can deploy them together as One Service instead of two independent services.

The solution is to **Mount Gradio Inside FastAPI**

5. Import Gradio in main.py

Open main.py

Add:

```
1 import gradio as gr
2 from gradio_app import demo
3
```

Python ▾

What Is demo?

In previous chapters, the Gradio UI was created using:

```
1 with gr.Blocks(...) as demo:
```

Python ▾

The variable **demo** represents the complete Gradio application.

6. Mount the Gradio Application

At the bottom of:

```
1 main.py
```

Plain Text ▾

add:

```
1 app = gr.mount_gradio_app(
2     app,
3     demo,
4     "/gradio"
5 )
```

Python ▾

This complete the mounting of Gradio onto the FastAPI application.

New Architecture

Before:

```
1 localhost:8000
2
3 FastAPI
4 ``
5
```

Plain Text ▾

and

```
1 localhost:7860
2
3 Gradio
4
5
```

Plain Text ▾

After:

```
1 localhost:8000
```

Plain Text ▾

FastAPI:

```
1 /
```

Plain Text ▾

Gradio:

```
1 /gradio
```

Plain Text ▾

Example

Swagger:

```
1 http://127.0.0.1:8000/docs
```

Plain Text ▾

Gradio:

```
1 http://127.0.0.1:8000/gradio
```

Plain Text ▾

One application. One deployment.

7. Test the Integrated Application

Run:

```
1 python -m uvicorn main:app --reload
```

Shell ▾

Open:

```
1 http://127.0.0.1:8000/docs
```

Plain Text ▾

Verify Swagger works.

Then open:

```
1 http://127.0.0.1:8000/gradio
```

Plain Text ▾

Verify Gradio works. Success.

8. Create requirements.txt

Render needs a list of required packages.

Activate the virtual environment.

Run:

```
1 pip freeze > requirements.txt
```

Shell ▾

A new file appears:

```
1 requirements.txt
```

Plain Text ▾

Example:

```
1 fastapi
2 uvicorn
3 sqlalchemy
4 gradio
5 openai
6 anthropic
7 python-jose
8 python-dotenv
9 ...
10
```

Plain Text ▾

Render will install these packages automatically.

9. Push the Project to GitHub

Create a new repository on GitHub.

Example:

```
1 fastapi-project
```

Plain Text ▾

Inside the project folder run:

```
1 git init
2
```

Shell ▾

Add files:

```
1 git add -A
2
3
```

Shell ▾

Commit:

```
1 git commit -m "Initial Commit"
2
```

Shell ▾

Add GitHub repository:

```
1 git remote add origin <your-github-repository>
```

Shell ▾

Push:

Shell ▾

```
1 git push -u origin main
2
```

Your project is now available on GitHub.

10. Create a Render Web Service

Log in to:

Plain Text ▾

```
1 https://render.com
2
```

Create:

Plain Text ▾

```
1 New Web Service
2
3
```

Connect:

Plain Text ▾

```
1 GitHub Repository
2
3
```

Select:

Plain Text ▾

```
1 fastapi-project
2
```

11. Configure Render

Use:

Plain Text ▾

```
1 Start Command
2
3
```

Shell ▾

```
1 uvicorn main:app --host 0.0.0.0 --port $PORT
```

This tells Render:

```
1 Run FastAPI
2 Listen on all network interfaces
3 Use Render's assigned port
4
```

Plain Text ▾

12. Deploy the Application

Render will:

```
1 Clone Repository
2   ↓
3 Install Packages
4   ↓
5 Run Uvicorn
6   ↓
7 Create Public URL
8
```

Plain Text ▾

Wait for deployment to complete.

13. Obtain the Render URL

Render generates a public URL.

Example:

```
1 https://fastapi-project-o8t0.onrender.com/
```

Plain Text ▾

Your URL will be different.

Verify FastAPI

Open:

```
1 https://fastapi-project-o8t0.onrender.com/docs
```

Plain Text ▾

Swagger should appear.

Verify Gradio

Open:

```
1 https://testapi-project-o7t0.onrender.com/gradio
```

Plain Text ▾

Gradio should appear.

14. Update gradio_app.py

Open:

```
1 gradio_app.py
```

Plain Text ▾

Add:

```
1 import os
```

Python ▾

Create:

```
1 API_BASE_URL = os.getenv(  
2     "API_BASE_URL",  
3     "https://fastapi-project-o8t0.onrender.com"  
4 )
```

Python ▾

Create:

```
1 USERS_API_URL = (  
2     f"{API_BASE_URL}/users"  
3 )
```

Python ▾

15. Test the Cloud Deployment

Open:

```
1 https://fastapi-project-o8t0.onrender.com/gradio  
2
```

Plain Text ▾

Try:

Plain Text ▾

```
1 Create User
2 Login
3 Upload File
4 AI Assistant
5 Business Assistant
```

Verify that:

- FastAPI endpoints respond correctly
- Gradio works correctly
- OpenAI and Claude integrations work
- MCP functionality works

Final Project Structure

Plain Text ▾

```
1 fastapi-project/
2 |
3 |— routers/
4 |— services/
5 |— tools/
6 |— mcp/
7 |— database/
8 |— uploads/
9 |— security/
10 |— dependencies/
11 |— config/
12 |
13 |— .env
14 |— requirements.txt
15 |— gradio_app.py
16 |— main.py
17 |
18 |— users.db
19
20
```

Key Takeaways

- ✓ Environment variables should be stored in `.env`
- ✓ FastAPI and Gradio can run as a single service
- ✓ `mount_gradio_app()` allows Gradio to run inside FastAPI
- ✓ Render can deploy FastAPI applications directly from GitHub
- ✓ `requirements.txt` is required for deployment
- ✓ Render automatically creates a public URL
- ✓ The Gradio UI is available through `/gradio`
- ✓ The completed AI Business Assistant can now be accessed from anywhere

Course Completion

Congratulations!

You have progressed from:

Python ▾

```
1 @app.get("/")
2
```

to a complete deployed AI platform:

Plain Text ▾

```
1 FastAPI
2   ↓
3 Database
4   ↓
5 Authentication
6   ↓
7 File Uploads
8   ↓
9 OpenAI
10  ↓
11 Claude
12  ↓
13 MCP Tools
14  ↓
15 MCP Server
16  ↓
17 AI Business Assistant
18  ↓
19 Cloud Deployment
20
21
```

You now have a complete end-to-end application that can be developed locally and deployed to the cloud.